CAS – AIS FS26

Agentic AI

From Language Models to Autonomous Agents

Building ML/AI Applications · ETH Zurich
CAS – AIS · FS 2026

What is Agentic AI?

An **AI agent** is a system that can:

- **Perceive** its environment
- **Reason** about what to do next
- **Act** using tools and APIs
- **Learn** from the results

Unlike a chatbot that answers one question, an agent **pursues goals over multiple steps**, deciding its own actions along the way.

Chatbot

Question $\rightarrow$ Answer

(single turn)

$\Downarrow$

Agent

Goal $\rightarrow$ Plan $\rightarrow$ Act $\rightarrow$ Observe $\rightarrow$ Adapt

(multi-turn, autonomous)

The Agent Loop

Every agent follows the same fundamental cycle:

The LLM is the **brain** – it reads observations and history, then outputs a structured tool call.

Anatomy of an Agent

LLM (The Brain)

Receives a **system prompt** (role, goals, rules) and **conversation history**. Outputs a **structured tool call**.

Memory / History

Running log of observations, actions, and results. Lets the agent **build context** and avoid repeating mistakes.

Tools

Concrete actions: `move()`, `talk()`, `search()`, `create_file()`, API calls, database queries.

Environment

The world the agent operates in: a game, a codebase, a database, a web app. Provides observations and responds to actions.

Why Agentic AI?

Real-world applications

- **Software engineering** – autonomous coding, debugging, testing
- **Customer support** – handle multi-step resolutions
- **Data analysis** – explore, query, visualize autonomously
- **Research** – literature review, experiment design
- **Operations** – supply chain, scheduling, monitoring

Why now?

- LLMs are finally **good enough at reasoning** to drive multi-step workflows
- **Tool use / function calling** standardized across providers
- Frameworks like **LangChain, CrewAI, Claude Agent SDK** make it accessible
- The gap between "chat" and "do" is closing fast

> *"The next frontier isn't smarter models – it's models that can **take action**."*

From Chatbot to Agent

	Chatbot	Agent
Interaction	Single turn Q&A	Multi-turn, goal-directed
Who decides?	Human gives each instruction	LLM decides its own next step
Tools	None (text only)	Can call functions, APIs, tools
Memory	Conversation context	Structured history + state tracking
Failure mode	Wrong answer	Wrong action (real consequences)
Output	Text response	Structured tool call $\rightarrow$ execution

The key shift: the LLM output is not the final answer – **it's an instruction that gets executed**.

CLASSIFIED – MISSION BRIEFING

The Hidden Layer

A Spy-Themed Agentic AI Exercise

AGENT LAMBDA · Isla Perdida · OVERFIT Base

Mission Overview

Objective

You are **Agent Lambda**. Infiltrate OVERFIT's base on Isla Perdida. **Collect 10 dossiers** of classified data before you run out of turns or health.

How to earn dossiers

- 1 each Collect dossier caches on the map
- 2 each Complete delivery quests for informants
- 3 each Destroy robots with the correct weapon
- 1 each Sell salvage at research facilities

Constraints

- **3 health points** – traps and failed robot encounters deal damage
- **100 turns max** – every action costs a turn
- **No map** – the agent only sees its current cell + adjacent cells

Your task

Implement the `think_llm()` function – the agent's **brain**. Everything else is provided.

Agent Tools

The agent has **4 tools** it can call. `scan()` runs automatically every turn.

move`(direction="north|south|east|west")`

Move one step. Triggers cell entry events.

talk`(message="your message")`

Converse with NPCs. Keywords trigger quest items.

collect`()`

Pick up items in current cell (dossiers, weapons, materials).

fabricate`(item="item name")`

Craft weapons or sell salvage at Forge/Lab.

scan`()` – *automatic, every turn*

Reports current position, adjacent cells, and what's visible. The agent receives this as an observation.

Tool Call Format

The LLM must output **exactly one** tool call in this format:

```
TOOL: move(direction="north")
TOOL: talk(message="Do you have any work for me?")
TOOL: collect()
TOOL: fabricate(item="Flamethrower")
```

The engine parses it with a simple regex:

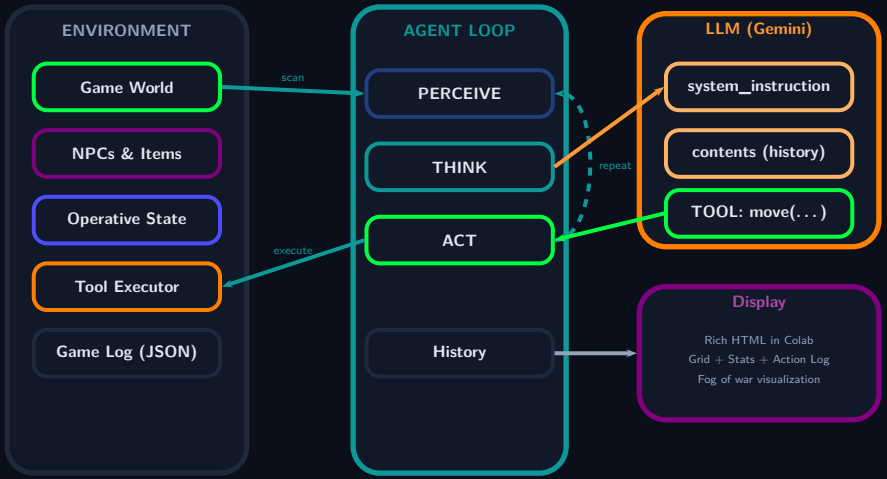
```python
def parse_tool_call(text: str) -> tuple[str, dict]:
    """Extract tool name and arguments from LLM output."""
    pattern = r'TOOL:\s*(\w+)\((.*?)\)'
    match = re.search(pattern, text, re.DOTALL)
    tool_name = match.group(1)

    args = {}
    for kv in re.finditer(r'(\w+)\s*=\s*"([^"]*)"', match.group(2)):
        args[kv.group(1)] = kv.group(2)

    return tool_name, args
```

The LLM can reason freely in text before the `TOOL:` line – only the tool call is extracted.

System Architecture



Three Mission Tiers

Start small, build confidence, tackle the full mission:

	Micro Mission	Training Mission	Full Mission
Grid	3×3	5×5	8×8
Dossiers needed	3	5	10
Max turns	30	50	100
NPCs	2	3	5+
Robots	1	1	2
Purpose	Learn the loop	Multi-step quests	Full strategy

Start with the **Micro Mission** – it's the fastest way to get a working agent. Then iterate.

Your Task: Implement `think_llm()`

This is the **only function** you need to write. It's the agent's brain.

```
def think_llm(operative, world, history, client) → str:
    """Decide the next action using an LLM.

    Steps:
    1. Build a system message (role, goals, tools, rules)
    2. Build a user message (status, history, observations)
    3. Call client.models.generate_content()
    4. Return response.text  →  must contain "TOOL: ..."
    """
    raise NotImplementedError("TODO: Implement this!")
```

Input

- `operative` – player state (health, dossiers, inventory, position)
- `world` – game world (grid, NPCs, quest flags)
- `history` – list of `{role, content}` dicts
- `client` – Gemini API client

Output

- A string containing **exactly one** tool call:

```
TOOL: move(direction="north")
```

The engine extracts and executes it.

MISSION STATUS: ACTIVE

Time to Code

Open the `Micro Mission` notebook and start building your agent.

Play manually → **Write think_llm()** → **Run & Debrief**

Good luck, Agent Lambda. The Hidden Layer awaits.